

Day 37: Remove Item from Cart

Day 37 — Remove Item from Cart

1. Day Number

Day 37

2. Topic Name

List remove() Method

Today you will learn how to remove an existing item from a Python list using:

```
remove()
```

You will also learn an important habit:

Check whether the item exists before trying to remove it.

3. Connection

Earlier, you learned how to **add products to a cart** using a list and `append()`.

For example:

```
Cart
↓
["Laptop", "Mouse", "Keyboard"]
```

Today you will do the opposite:

```
Add item
  ↓
append()

Remove item
  ↓
remove()
```

So your cart can now support both:

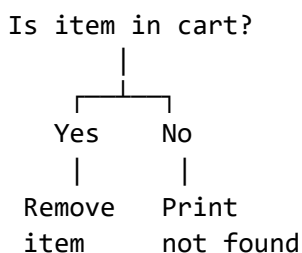
```
Product
  ↓
Add to Cart
  ↓
Cart List
  ↓
Remove from Cart
```

4. Important Topics

The important concepts for today are:

- list
- remove()
- in
- if-else
- function
- function parameter
- searching before removing

The most important pattern is:



5. Foundational Notes

What is a list?

A list stores multiple values together.

Example:

```
cart = ["Laptop", "Mouse", "Keyboard"]
```

Here:

Index	Item
0	Laptop
1	Mouse
2	Keyboard

What does remove() do?

The remove() method removes a particular **value** from a list.

Example:

```
fruits = ["apple", "banana", "mango"]

fruits.remove("banana")
```

After removing:

```
["apple", "mango"]
```

Notice that we provide the **value**:

```
remove("banana")
```

not the index.

remove() vs append()

You have already used:

```
append()
```

to add something.

Example:

```
cart.append("Mouse")
```

Today:

```
cart.remove("Mouse")
```

Conceptually:

```
append()
  |
  v
["Laptop"]
  |
  + Mouse
  |
  v
["Laptop", "Mouse"]

remove()
  |
  v
["Laptop", "Mouse"]
  |
  - Mouse
  |
  v
["Laptop"]
```

Why search before removing?

Suppose:

```
cart = ["Laptop", "Mouse"]
```

and you try:

```
cart.remove("Keyboard")
```

But "Keyboard" is not inside the list.

Python would produce an error similar to:

```
ValueError
```

Therefore, for today's exercise, first check:

```
if item in cart:
```

and only then use:

```
remove()
```

The logic becomes:

```
item
|
v
Is item in cart?
|
+----- Yes -----> remove item
|
+----- No -----> item not found
```

The in operator

Python's `in` operator checks whether something exists inside a collection.

Example:

```
colors = ["red", "blue", "green"]

if "blue" in colors:
    print("Found")
```

Here:

```
"blue" in colors
```

evaluates to:

```
True
```

But:

```
"yellow" in colors
```

evaluates to:

```
False
```

This is exactly what we need before removing an item.

6. Easy Example

Consider a simple list of students:

```
students = ["Amit", "Rahul", "Neha"]

name = "Rahul"

if name in students:
    students.remove(name)
    print("Student removed")
else:
    print("Student not found")
```

After the removal:

Before:

```
["Amit", "Rahul", "Neha"]
```

```
    remove Rahul
      ↓
```

After:

```
["Amit", "Neha"]
```

The important idea is not the student example itself.

The pattern is:

```
Check
 ↓
Found?
 ↓
Remove
```

You will apply the same idea to a shopping cart.

7. Problem Statement

Create a cart containing some item names.

For example, conceptually:

```
Laptop
Mouse
Keyboard
```

Then:

1. Ask the user to enter an item name.
2. Check whether that item exists in the cart.
3. If the item exists:
 - remove it from the cart.
4. Otherwise:
 - print "Item not found".
5. Show the updated cart.

Example flow:

```
Cart
["Laptop", "Mouse", "Keyboard"]

User enters:
Mouse

      ↓

Is Mouse in cart?
      ↓
      Yes
      ↓
Remove Mouse
      ↓

Updated cart
["Laptop", "Keyboard"]
```

8. Concepts Used

List

Used to store cart items.

Conceptually:

```
cart = [...]
```

User Input

Used to ask which item should be removed.

```
input()
```

Membership Check

Use:

in

to determine whether the requested item exists.

Conceptually:

item in cart

Condition

Use:

if

for the successful case.

And:

else

for the item-not-found case.

remove()

Used to remove the matching value.

cart.remove(item)

Function

Instead of placing all logic directly in the main program, today's suggested approach is to put the removal logic inside a function.

Conceptually:

```
Main Program
  |
  | cart + requested item
  v
Remove Function
  |
  | checks item
  |
+---- found ----> remove
  |
+---- not found -> message
```

9. Thought Process

Before writing Python, think through the problem step by step.

Step 1: What data do I have?

A cart containing item names.

```
["Laptop", "Mouse", "Keyboard"]
```

Step 2: What information do I need from the user?

The name of the item they want to remove.

Example:

```
Mouse
```

Step 3: Can I immediately call remove()?

Not safely.

First ask:

```
Does this item exist in the cart?
```

Step 4: How do I check that?

Use:

```
in
```

Conceptually:

```
item in cart
```

Step 5: What happens if the item exists?

Use:

```
remove()
```

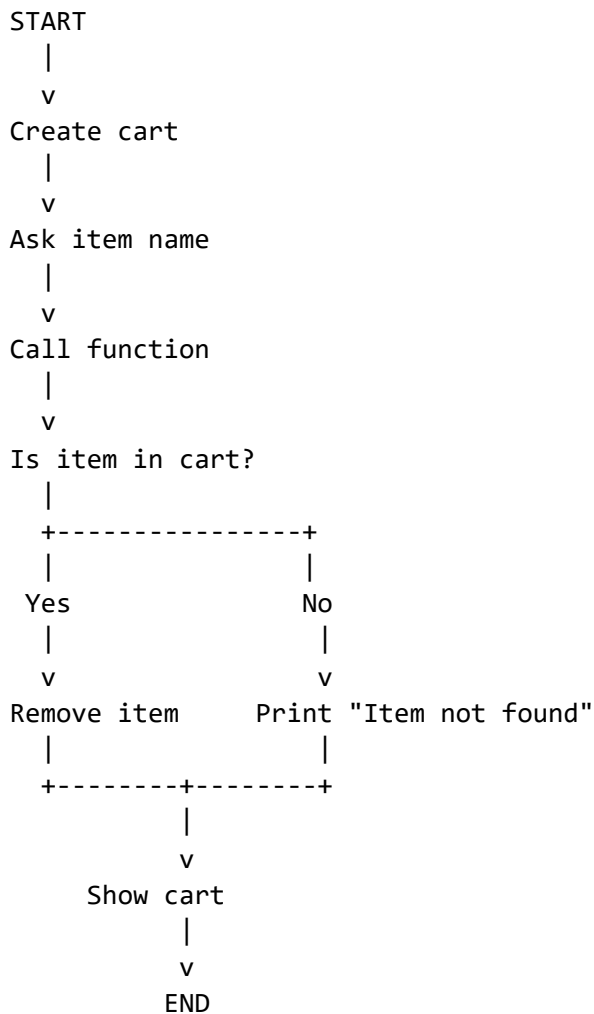
Step 6: What happens if it does not exist?

Do not call remove().

Instead print:

```
Item not found
```

Complete thinking flow



10. Pseudocode

Do not worry about exact Python syntax yet.

```
FUNCTION remove_item(cart, item)
```

```
    IF item exists in cart
        remove item from cart
        print item removed
    ELSE
        print item not found
```

```
END FUNCTION
```

```
Create cart with item names
```

```
Ask user which item should be removed
```

```
Call remove_item function
```

Print updated cart

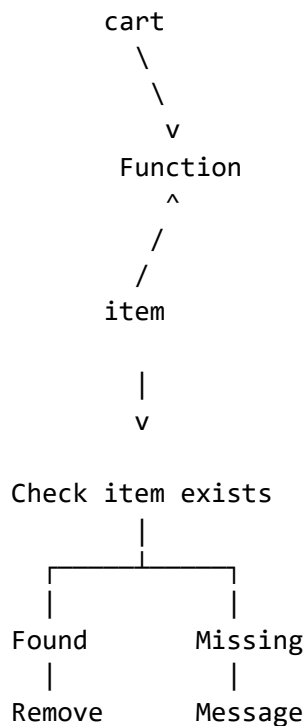
Notice that the pseudocode answers the important questions before coding:

What data?
What condition?
What happens when true?
What happens when false?

11. Suggested Solving Approach — Functional Approach

Use a small function responsible for removing an item.

Think of the function like this:



A possible function structure is:

```
def remove_item(cart, item):  
    # check whether item exists  
  
    # if found:  
        # remove item  
  
    # otherwise:  
        # print item not found
```

This is intentionally incomplete.

Your job is to fill in the condition and removal logic.

12. Easy Edge Cases

Edge Case 1: Item Not Found

Cart:

```
["Laptop", "Mouse", "Keyboard"]
```

User enters:

```
Monitor
```

Since "Monitor" is not present:

```
Item not found
```

The cart should remain unchanged:

```
["Laptop", "Mouse", "Keyboard"]
```

Edge Case 2: Empty Cart

Cart:

```
[]
```

User enters:

```
Mouse
```

There is nothing to remove.

Therefore:

```
Item not found
```

This is useful because your same condition can naturally handle an empty cart.

```
[]  
|  
| Search Mouse  
v  
Not found
```

You do not need complicated special logic for this beginner exercise.

Small Extra Observation: Duplicate Items

Suppose:

```
["Mouse", "Keyboard", "Mouse"]
```

If you use:

```
remove("Mouse")
```

Python removes the **first matching occurrence**.

Result:

```
["Keyboard", "Mouse"]
```

You do not need to solve duplicate removal today, but it is useful to know how `remove()` behaves.

13. Expected Input

Your program can start with a predefined cart, so the main user input is an item name.

Example:

```
Enter item to remove: Mouse
```

Another example:

```
Enter item to remove: Monitor
```

The input is a string.

Conceptually:

```
User
|
| "Mouse"
v
Program
```

14. Expected Output

Case 1 — Item Exists

Suppose the cart initially contains:

```
["Laptop", "Mouse", "Keyboard"]
```

Input:

```
Enter item to remove: Mouse
```

Expected behavior:

```
Item removed
Updated cart: ['Laptop', 'Keyboard']
```

Case 2 — Item Does Not Exist

Input:

```
Enter item to remove: Monitor
```

Expected behavior:

```
Item not found  
Updated cart: ['Laptop', 'Mouse', 'Keyboard']
```

Case 3 — Empty Cart

Cart:

```
[]
```

Input:

```
Enter item to remove: Mouse
```

Expected behavior:

```
Item not found  
Updated cart: []
```

15. Hint Only

Start by creating something similar to:

```
cart = ["Laptop", "Mouse", "Keyboard"]
```

Then get the item from the user:

```
item = input(...)
```

Inside your function, think about this question:

```
How can I write:  
  
"if item exists inside cart"
```

You learned the keyword you need today:

```
in
```

Then, only inside the successful condition, use:

```
cart.remove(...)
```

Your core logic should mentally look like:

```
IF requested item is inside cart
    remove requested item
ELSE
    print item not found
```

Do not call `remove()` before checking that the item exists.